# Coding Challenge - Live Staffing Dashboard

## Context and objective

An event manager needs three workers for a shift and sends them assignment invitations. Workers can accept or decline. Workers who accept can check in when they arrive and check out when they finish. The manager needs to see staffing and attendance changes as they happen, without refreshing the page. Build a small application from scratch with a worker view and a manager dashboard. They should work in separate browser sessions and stay consistent through a shared backend and database.

## What to build

Start with one shift requiring three workers and three named workers, initially uninvited. Include a shift title, date, start/end time, and location.

- **Worker view:** select a worker, see their invitation, accept or decline, and check in or out if accepted. A simple selector is enough; login is not required.
- **Manager dashboard:** send assignment invitations and show shift details, each worker's current status, check-in/out timestamps, filled and remaining places, and counts of workers currently checked in and checked out.

**Invited** → **Accepted / Declined** → **Checked in** → **Checked out** (only Accepted continues to check-in).

Use these two status categories:

| **Category** | **Statuses** | **Meaning** |
|---|---|---|
| Participation | Uninvited, Invited, Accepted, Declined, Cancelled | Whether the worker has been invited and agreed to work the shift. |
| Attendance | Not started, Checked in, Checked out | Whether the worker has arrived or finished. Initially Not started. |

The manager sends an invitation; the worker accepts or declines. Only an Accepted worker with attendance Not started can check in. Only a Checked in worker can check out; Checked out is final for this shift. An Accepted worker may cancel before checking in, and a Cancelled worker may accept again. Participation stays Accepted during check-in and check-out. Count Accepted workers toward filled places, including those Checked in or Checked out. All dashboard counts must agree with the worker list.

## Live updates and reliability

- Invitations and worker actions update the manager dashboard and other affected sessions automatically, without refreshing.

- Save and validate changes through a backend and database. Refreshing must preserve saved statuses and timestamps.
- Repeated clicks or retries must not duplicate records, change recorded timestamps, or break counts. Give useful feedback when an action fails.
- Indicate when live updates are unavailable and catch up automatically when connectivity returns.

Choose your update mechanism and explain why. Measure update delay and reconnection recovery time; simple measurements with a brief explanation are sufficient.

## Technology and submission

**You must use AI tools during development.** There are no restrictions on the technologies, programming languages, databases, or infrastructure you choose for this exercise. You are free to use any technology stack you believe is appropriate for solving the problem.

We are interested in understanding not only how you implement the solution, but also the reasoning behind your technical decisions.

Submit a repository with setup/reset instructions, a few meaningful tests, and a concise README covering:

- *Architecture and design:* Explain your overall architecture, data model, update mechanism, and key design trade-offs.
- *Technology choices:* Describe the technologies, frameworks, databases, and infrastructure you selected. Most importantly, explain why you chose them and what alternatives you considered.
- Testing and measurements: Explain how you tested status transitions, illegal actions, counts, repeated actions, and connection recovery. Include any relevant performance measurements or observations.
- Challenges and learnings: Describe the challenges you encountered, how you addressed them, and what you learned during implementation.
- Limitations and improvements: Highlight any known limitations and explain what you would improve or change if you had more time.

## Creativity

Your own visual design and optional features are welcome. Explain what they improve. Keep the core experience reliable and easy to demonstrate.

## Follow-up discussion (15-20 mins)

Demonstrate the application, walk through the code, and discuss your design decisions, reliability, testing, challenges, what you learned, and AI use. Explain what you did to **improve code quality, keep the implementation lean, and make it faster or more efficient**. Support performance claims with measurements where practical. Be ready to investigate or make a small change live.
You will be evaluated on **code quality, system design, reliability, simplicity, performance, and ownership of your AI-assisted work**. We value clear structure, minimal unnecessary complexity, and optimizations you can explain and justify.